

First Hibernate Application

Hibernate is a popular ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java classes to database tables. Below are the key components and steps for creating your first Hibernate application.

1. Model Class: Student

- **Annotations Used:**

- `@Entity`: Marks the class as an entity, meaning it will be mapped to a database table.
- `@Table(name = "StudentTable")`: Specifies the name of the database table.
- `@Id`: Marks the primary key of the entity.
- `@Column(name = "SID")`: Maps the field to the corresponding column in the database.

- **Code Example:**

```
@Entity
@Table(name="StudentTable")
public class Student {

    @Id
    @Column(name="SID")
    private Integer sid;

    @Column(name="SNAME")
    private String sname;

    @Column(name="SCITY")
    private String scity;

    public Student() {
        System.out.println("Zero Param Constructor for Hibernate");
    }
}
```

```
}

// Getters and Setters

}
```

Notes: If you do not specify the table name or column names in Hibernate using the `@Table` and `@Column` annotations, Hibernate will apply default naming conventions:

1. Default Table Name:

- By default, the table name will be the same as the class name of the entity.
- For example, if your entity class is named `Student`, the default table name will be `Student`.

2. Default Column Names:

- The column names will be the same as the field names in your entity class.
- For example, if you have a field `private String sname;`, the default column name will be `sname`.

2. Hibernate Configuration File (`hibernate.cfg.xml`)

● **Important Properties:**

- `hibernate.connection.driver_class`: Specifies the JDBC driver class.
- `hibernate.connection.url`: The JDBC URL to connect to the database.
- `hibernate.connection.username` and `hibernate.connection.password`: Credentials for the database.
- `hibernate.dialect`: Specifies the SQL dialect Hibernate should use (e.g., `org.hibernate.dialect.MySQL8Dialect`).
- `hibernate.hbm2ddl.auto`: Specifies how Hibernate should handle database schema creation/update. Values can be `create`, `update`, `validate`, etc.
- `hibernate.show_sql`: If set to `true`, Hibernate will log the generated SQL to the console.

- hibernate.format_sql: Formats the SQL output in the console for better readability.
- <mapping class="com.telusko.model.Student"/>: Specifies the class to be mapped to the database.

- **Code Example:**

```
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/yourDatabaseName</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">root123</property>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
    <property name="hibernate.hbm2ddl.auto">create</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <mapping class="com.telusko.model.Student"/>
  </session-factory>
</hibernate-configuration>
```

3. Main Class for Hibernate Operations

- **Steps Involved:**

1. **Configuration Object:** Loads the Hibernate configuration file.
2. **Build SessionFactory:** Creates a SessionFactory from the configuration object.
3. **Open Session:** Opens a new session from the SessionFactory.
4. **Begin Transaction:** Begins a transaction for database operations.

5. **Perform Operations:** In this example, create a Student object, set its properties, and save it to the database.
 6. **Commit Transaction:** Commits the transaction to persist the changes.
 7. **Close Session:** Closes the session after the operation is complete.
- **Code Example:**

```
public static void main(String[] args) {  
    // Step 1: Configuration Object  
    Configuration config = new Configuration();  
    config.configure();  
  
    // Step 2: Create SessionFactory Object  
    SessionFactory sessionFactory = config.buildSessionFactory();  
  
    // Step 3: Get the Session Object from Session Factory  
    Session session = sessionFactory.openSession();  
  
    // Step 4: Begin the Transaction within Session  
    Transaction transaction = session.beginTransaction();  
  
    // Step 5: Perform Operation  
    Student student = new Student();  
    student.setSid(1233);  
    student.setSname("Arjun");  
    student.setScity("Ballia");  
  
    session.save(student);  
  
    // Step 6: Commit Transaction  
    transaction.commit();  
  
    // Step 7: Close the Session  
    session.close();  
}
```

```
}
```

Note on Configuring hibernate.cfg.xml File

- **Default Behavior:** When configuring Hibernate, the framework automatically looks for a configuration file named hibernate.cfg.xml in the classpath. You can load this configuration using:

```
Configuration config = new Configuration();  
config.configure();
```

- **Custom Configuration File Name:**
 - If your configuration file is named something other than hibernate.cfg.xml (e.g., hib.cfg.xml), you must explicitly specify this file name when configuring Hibernate.
 - Example:

```
Configuration config = new Configuration();  
config.configure("hib.cfg.xml");
```

Key Points:

- **Default:** config.configure(); assumes the file is named hibernate.cfg.xml.
- **Custom:** Use config.configure("custom-file-name.xml"); if you have renamed your configuration file.

Output:

```
mysql> use telusko_db  
Database changed  
mysql> select * from Student;  
ERROR 1146 (42S02): Table 'telusko_db.student' doesn't exist  
mysql> show tables;  
+-----+  
| Tables_in_telusko_db |  
+-----+  
| personinfo           |  
| studenttable         |  
+-----+  
2 rows in set (0.01 sec)  
  
mysql> select * from studenttable;  
+-----+  
| sid | scity | sname |  
+-----+  
| 123 | Ballia | Shiva |  
| 1233 | Ballia | Arjun |  
+-----+  
2 rows in set (0.00 sec)
```

